Week 1 Level 2: Intermediate

Welcome Back, Python

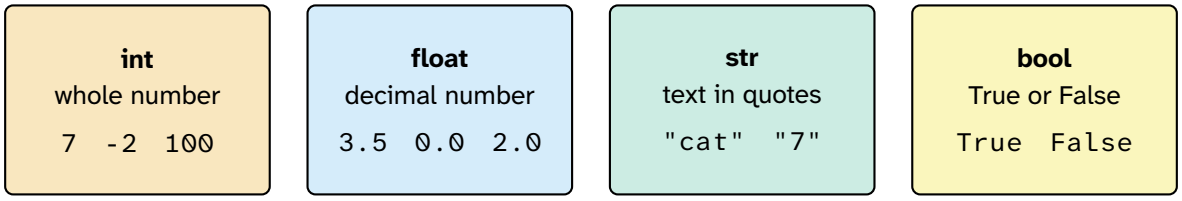
Every value has a type, and f-strings glue values into friendly messages.

What you will learn

- The four basic **types** of value: whole numbers, decimal numbers, text and True/False.
- How to build messages with **f-strings**.
- How to turn typed answers into numbers with `int()`.
- How to make smarter choices with `and`, `or` and `not`.

The big idea: every value has a type

Python sorts every value into a **type**. The type decides what Python can do with it. Here are the four types you will use most:



Quotes are the big clue. 7 is a number, but `"7"` is text that happens to show a 7. You can ask Python for the type of anything. Type this in the Thonny **Shell** (after >>>):

Try it in the Shell

```
type(7)        # <class 'int'>
type("7")      # <class 'str'>
type(3.5)      # <class 'float'>
type(True)     # <class 'bool'>
```

Bug hunt: text plus a number

Try `"5" + 5` in the Shell. Python shows a red **error message**. Always read it from the **bottom line up**:

Output

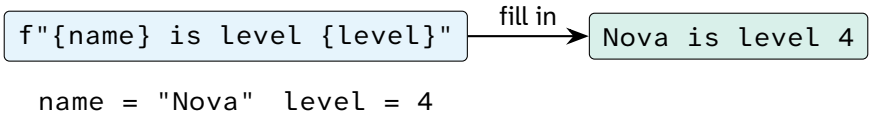
```
TypeError: can only concatenate str (not "int") to str
```

"Concatenate" means "glue together". Python can glue text to text, but not text to a number. This matters because **`input()` always gives back text**, even when you type a number. Turn it into a whole number with `int()`:

```
level = int(input("Level? "))      # the player types 4
print(level + 1)                   # 5
```

f-strings: fill in the blanks

Put an `f` in front of the quotes, and Python fills every pair of curly brackets with a value. It works with any type, so you never have to worry about text plus a number.



Smarter choices: and, or, not

Word	True when...	Game example
`and`	**both** sides are true	`has_key and level >= 3`
`or`	**at least one** side is true	`level >= 10 or has_map`
`not`	the opposite	`not game_over`

Each side needs a full comparison: write `energy >= 5 or energy == 1`, not `energy >= 5 or 1`.

Tip: Hero names, not real names

Your real name, school, address and birthday are **personal information**. Keep them private. In our programs you always type a made-up **hero name** or nickname.

Exercise 1.1: Hero Card

Folder: `exercises/week01/ex-hero-card`. Open `starter.py` and press **Run** (F5).

1. It crashes! Read the error from the bottom up. Write the last line here:

   _____

2. TODO 1: turn the level into a number with `int()`. Run again.
3. TODO 2–5: add the missing lines with f-strings: power, power points, health (`level * 12.5`) and "Legend?".
4. Which type is each value? Write int, float, str or bool:

   `hero_name` _____ `power_points` _____ `health` _____ `is_legend` _____

5. Check your work: open `check_my_work.py` and press **Run**.

Exercise 1.2: Mood Meter

Folder: `exercises/week01/ex-mood-meter`. The program asks for your energy (0–10) and whether you had a snack, then draws a meter. Finish `mood_for()`:

- "Super charged": energy 8 or more **and** a snack.
- "Ready to play": energy 5 or more **or** a snack.
- "Recharge time": everything else.

Before you run it, **predict** the mood. Then run and check.

energy	snack?	my guess	the program said
9	yes		
9	no		
2	yes		
3	no		

Last step: use `if not had_snack:` to print a friendly tip. Then check your work with `check_my_work.py`.

Vocabulary box

type The kind of value: int, float, str or bool. It decides what Python can do with it.

int A whole number, like 7 or -2.

float A number with a decimal point, like 3.5 or 2.0.

str (string) Text inside quotes, like `"cat"` or `"7"`.

bool True or False. Comparisons like `level >= 3` give a bool.

f-string Text with an f in front. Curly brackets inside are filled with values.

TypeError An error that means “these two types do not go together”.

personal information Facts that say who you really are or where to find you. Keep it private.

Reflection

Which value card or error surprised you today? Why?

Make up a game rule that uses `and` or `or`:

Week 1 Level 2: Intermediate

Welcome Back, Python

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Type detective (paper, 3 minutes)

Write the type of each value: **int**, **float**, **str** or **bool**.

`42`		`"42"`	
`4.2`		`False`	
`"False"`		`-8`	

★ Core (everyone): Dragon Door (Thonny, 10 minutes)

Open `exercises/week01/hw-dragon-door/starter.py`. The dragon door opens if:

- you have the golden key **and** you are level 3 or more, **or**
- you are level 10 or more (legends do not need keys).

1. TODO 1: change the `if` line in `door_opens()`. Brackets help: `(has_key and level >= 3) or level >= 10`
2. TODO 2: make the welcome message an f-string that says the hero name.
3. Run it three times and check what happened:

   key yes, level 5: opens ☐ shut ☐
   key yes, level 2: opens ☐ shut ☐
   key no, level 12: opens ☐ shut ☐

4. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Predict, then check

Write what each line prints. Then type it into the Thonny Shell to check.

```
print(3 > 2 and 5 > 9)
print(3 > 2 or 5 > 9)
print(not 3 > 2)
print("7" + "7")
```

```
print(int("7") + 7)
```

Creative (make it yours): Design a hero

Invent a hero for a game (a made-up name only). Draw them in the box and write a hero card with at least one value of each type: a str, an int, a float and a bool. Next week you can type it into the Hero Card program.

Self-check: tick each box when it is true

- ☐ I can name the four types and give an example of each.
- ☐ I know that `input()` gives back text, and how `int()` fixes that.
- ☐ My Dragon Door opens for the right heroes and stays shut for the others.
- ☐ I used a made-up hero name, not my real name.